Week 1 - Domain 1: SDLC Automation - AWS CodeCommit

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Covers: repos, authentication, IAM, branch protection, triggers, notifications, EventBridge, cross-account access

Part 1: AWS CodeCommit

1.1 What CodeCommit is

- **Managed private Git hosting.** AWS runs it, so there are no servers to manage, it scales on its own, and it is highly available.
- Your code stays inside AWS and is controlled by **IAM**, logged by **CloudTrail**, and encrypted by **KMS**.
- **Encryption at rest:** always on, using KMS. The default is the AWS-managed key `aws/codecommit`; you can choose a **customer-managed key** instead.
- **Encryption in transit:** HTTPS or SSH.
- **Status note:** AWS stopped onboarding new customers in July 2024 and later restored full availability (check the current docs). It is **still in scope for DOP-C02**.

1.2 Authentication: how you connect

Method	Who uses it	Key points
HTTPS Git credentials	IAM users	A static username and password generated in IAM. Simplest, but long-lived.
SSH keys	IAM users	Upload a public key to your IAM user; the SSH key ID is the username.
git-remote-codecommit (GRC)	IAM users, roles, SSO / federated users	Uses **temporary credentials** from an AWS CLI profile. URL: `codecommit::us-east-1://profile@repo`. **Recommended.**
AWS CLI credential helper	Roles and federated users	Older way to use temporary credentials over HTTPS.

Exam traps:

- The **root user cannot** use Git credentials or SSH keys with CodeCommit.
- Federated, SSO, or cross-account users must use **GRC or the credential helper**, because they have no IAM user to hold Git credentials or SSH keys.

1.3 Authorization: IAM only

- CodeCommit has **no resource-based policies on repositories** (unlike S3 bucket policies). Very common exam point.
- All access is controlled by **identity-based IAM policies**. Repository ARN: `arn:aws:codecommit:us-east-1:111122223333:MyRepo`.
- AWS managed policies:
  - `AWSCodeCommitFullAccess` - everything, including deleting repositories.
  - `AWSCodeCommitPowerUser` - everything **except deleting repositories**. The usual developer policy.
  - `AWSCodeCommitReadOnly` - read-only access.

1.4 Branch protection

There is no "protected branch" checkbox like GitHub has. You combine two things.

a) An IAM Deny policy on specific branches - the condition key `codecommit:References` matches the branch name. Attach it to the developers' group:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": [
      "codecommit:GitPush",
      "codecommit:DeleteBranch",
      "codecommit:PutFile",
      "codecommit:Merge*"
    ],
    "Resource": "arn:aws:codecommit:us-east-1:111122223333:MyRepo",
    "Condition": {
      "StringEqualsIfExists": { "codecommit:References": ["refs/heads/main"] },
      "Null": { "codecommit:References": "false" }
    }
  }]
}
```

- The `Null` block makes the Deny apply only when a branch reference is present in the request.
- Leads and the pipeline role do **not** get this policy, so they can still merge.

b) Pull request approval rules

- An **approval rule** applies to a single pull request.
- An **approval rule template** is reusable: associate it with repositories and it applies to new pull requests targeting the branches you list.
- Example: "2 approvals from members of the `SeniorDevs` IAM role" before a merge is allowed. Approvers can be IAM users or an **approval pool** of roles with wildcards.

Typical exam answer: "Prevent direct pushes to main and require 2 reviewers" = **IAM Deny using** `codecommit:References` + **approval rule template**.

1.5 Triggers, notifications, and EventBridge

The most-tested part of CodeCommit. Learn the differences:

Feature	Events it reacts to	Targets	Use it when
Triggers (repository setting)	Only repository changes: push to a branch, create / delete branch or tag	SNS, Lambda (max 10 triggers per repository)	A simple action when someone pushes
Notification rules (CodeStar Notifications)	Pull requests (created, updated, merged), comments, approvals, branches and tags	SNS, Slack / Teams via AWS Chatbot (Amazon Q Developer in chat applications)	Telling **people** what happened
Amazon EventBridge	**All** CodeCommit events: referenceCreated / Updated / Deleted, pullRequestCreated, PR state changes, comments	Any EventBridge target: Lambda, CodeBuild, Step Functions, SQS, CodePipeline	Automation, especially for **pull request events**

Exam rules of thumb:

- **Run a build or test when a pull request is created** (CodeBuild posts the result as a PR comment): use **EventBridge**. Triggers cannot see pull request events.
- **Start CodePipeline from CodeCommit:** use an **EventBridge rule** (default and recommended). Periodic polling is legacy.
- **Notify a Slack channel about pull requests:** use **notification rules** with Chatbot.
- **Copy a repository to another Region or account:** there is **no built-in replication**. Use EventBridge to run CodeBuild or Lambda that does `git push --mirror`.

1.6 Cross-account access

Scenario: the repository is in **Account A**, the developers are in **Account B**.

1. In **Account A**, create an IAM role, for example `CodeCommitCrossAccountRole`:
   - **Trust policy:** allows Account B (`arn:aws:iam::<AccountB>:root`).
   - **Permissions policy:** CodeCommit access to that repository only.

2. In **Account B**, grant the developers' group `sts:AssumeRole` on that role's ARN.
3. Developers configure an AWS CLI profile that uses the role, then clone with GRC:

```
git clone codecommit::us-east-1://crossaccount-profile@MyRepo
```

Remember: there is no repository policy, so cross-account access **always means assuming a role**.

Cross-account CodePipeline (pipeline in a "Tools" account, repository in a "Dev" account) needs:

- A role in the source account that the pipeline can assume.
- A **customer-managed KMS key** for the artifact bucket (the default key cannot be shared across accounts).
- An S3 bucket policy that lets the other account's role read and write artifacts.

1.7 Other exam points

- **Monitoring:** CloudTrail records CodeCommit API calls, including `GitPush` and `GitPull`. Use it for "who deleted the branch?" questions.
- **Code review:** attach **CodeGuru Reviewer** to a repository for automatic comments on pull requests.
- **Migrating from GitHub or Bitbucket:** `git clone --mirror`, then `git push --mirror` to CodeCommit.
- **Large files:** files over about 6 MB can't be added through the console or API; use Git LFS or store them in S3.
- **Permissions for targets:** an SNS topic policy allowing `codestar-notifications.amazonaws.com`, and a Lambda resource policy allowing `codecommit.amazonaws.com`.

1.8 Hands-on lab (about 1.5 hours)

1. Create a repository called `dop-lab-repo`.
2. Install git-remote-codecommit (`pip install git-remote-codecommit`) and clone with `git clone codecommit::us-east-1://dop-lab-repo`.
3. Create an IAM group `Developers` with `AWSCodeCommitPowerUser` and the Deny-on-main policy. Check that pushing to `main` fails and pushing to `feature/x` works.
4. Create an approval rule template requiring 1 approval for `main` and associate it with the repository. Try to merge a pull request without approval.
5. Add a trigger that invokes a Lambda function on push; log the event.
6. Create an EventBridge rule for `pullRequestCreated` that publishes to an SNS email topic.
7. Create a notification rule that sends pull request comments to SNS.
8. Clean up: delete the repository, Lambda function, SNS topic, and rules.

1.9 Check yourself

1. A developer signs in through IAM Identity Center and needs to clone a repository. Which authentication method should they use?
2. You must run unit tests automatically when a pull request is created and post the result as a comment. Trigger or EventBridge, and why?
3. Can you grant another account access by attaching a policy to the repository? If not, what do you do?

Answers:

1. **git-remote-codecommit** (or the credential helper) - federated users have no IAM user for Git credentials or SSH keys.
2. **EventBridge.** Triggers only fire on repository events (pushes, branch changes), not pull request events. The rule starts CodeBuild, and Lambda posts the result with `PostCommentForPullRequest`.
3. **No.** CodeCommit has no repository policies. Create a role in the repository's account that trusts the other account, and have the users assume it.

1.x More exam scenarios (added in the gap review)

- **Only from the corporate network or VPC:** IAM policy or **SCP Deny** on `codecommit:*` unless `aws:SourceIp` is the corporate range or `aws:SourceVpce` is the company's CodeCommit VPC endpoint. CodeCommit has no security groups and no repository policies.

- **Large binary files** (ML models, media): store them in **S3** (versioned) or Git LFS and keep only a reference in the repository. Git history with big binaries slows every clone, and the console / API limit is about 6 MB per file.